EpiScope: A Big Data Platform for Disease Surveillance, Forecasting and Response Planning in Bangladesh

Group [TEAMMATE: number]: Nishadul Islam Shezan (14), Tahsin Ahmed Rafi (13), Rimjhim Dey (39), Samira Sultana (21 [TEAMMATE: confirm ID])
CSE 4345 Big Data Analytics, Premier University, Chittagong
Repository: https://github.com/Shinzuu/episcope Video: [link added at v1.0]

***Abstract*—EpiScope turns routine health, climate and logistics data into district-level early warnings, one to four week forecasts, an allocation of a fixed national stock and a seasonal preparedness calendar. It is built as a Spark and Delta Lake medallion lakehouse over a synthetic encounter feed of more than 25 million records that takes its population, geography and weather from real public data. At full scale (34 million encounters) the pipeline runs in about 25 minutes on a 16 GB laptop. On 35 planted outbreaks, EARS C2 detects 33 with a median delay of eight days and fewer than two false alerts per district-year; the negative-binomial forecasts beat the seasonal-naive baseline with a relative weighted interval score of 0.84; and at half the money needed, the allocation meets 68% of the need against 34% for a population-based split. Only aggregated, small-cell suppressed data reaches the D3.js dashboard.**

I. Introduction and Problem Analysis

Bangladesh carries a heavy and strongly seasonal load of infectious disease. The 2023 dengue season was the worst on record: WHO counted 308,167 cases and 1,598 deaths up to the end of November [1]. Data that could warn earlier already exists, but it sits in separate systems that differ in format, timeliness and quality, so most analysis happens after hospitals are already full.

The course's complex engineering problem (CEP) casts the Ministry of Health (MoH) as the client. It asks for a prototype that, for each of the 64 districts, (i) detects unusual rises in dengue, diarrhoea, influenza-like illness (ILI) and malaria, (ii) forecasts cases and hospital demand one to four weeks ahead, (iii) allocates a limited national stock with a linear programme (LP; in the end a small mixed-integer one), and (iv) gives a preparedness calendar for the next year. The users are not data scientists, the data is late and incomplete, no patient may be exposed, and the system must run on a 16 GB laptop.

II. CEP Attributes and Conflict Resolution

A. Attributes P1 to P7

- **P1 depth of knowledge.** Distributed processing and storage (Spark, Delta, partitioning, broadcast joins), keyed hashing built from `sha2`, surveillance statistics (EARS C2, CUSUM), count regression (NB-GLM), linear programming and visual encoding in D3.

TABLE I
Stakeholders and what the design does for each

Stakeholder	Need, and where the design meets it
MoH policymakers	Defensible allocation: LP against a population baseline, equity floor α
Epidemiologists	Transparent methods: EARS C2 and CUSUM, thresholds from a sweep, artefact handling
District officers	Early local warning: ranked alert table, district view
Hospital managers	Bed and supply demand with 80% intervals
Vector control	Hotspots by district and week; upazila withheld for privacy
Patients, public	Keyed hash, age bands, suppression of counts 1 to 4
Privacy regulator	Line-level data never leaves the lake; only aggregates reach the browser

TABLE II
Risk register from the proposal, with the outcome

Risk	Outcome
Generator too slow	Retired: 34.1 million SF=1 encounters in under a minute (Section IV)
SF=1 over 45 minutes	Met: 25 to 28 minutes from a fresh clone (Table VII)
Hidden drop fails	Retired: the drill on the final code passed every check (Section IV)
Reporting artefact raises false alerts	Mitigated: onset-week counts, silent weeks missing; a test checks it
Small-cell leak through totals	Mitigated: audit finds no leak; a leak by differencing between district bundles was found in review and closed
Weather API rate limit	Retired: downloaded once, processed data committed
Uneven commits	Occurred: three members were away in the final phase (Appendix B)
Allocation never tested on real need (added)	Occurred and fixed: stock and LP faults (Section VI-C)

- **P2 conflicting requirements.** The five conflicts below, each settled with a number.
- **P3 depth of analysis.** Thresholds come from a sweep against injected outbreaks, forecasts from a 52-origin backtest against a baseline, and α from a curve over budgets.
- **P4 unfamiliar issues.** Late records (MERGE on the latest report), a facility that goes silent and back-fills (onset-week counts, missing rather than zero), and the same patient at two facilities (DQ-4).
- **P5 beyond standard codes.** ICD-10 three-character buckets, WHO IDSR [2] for the diseases and the weekly cadence, and our own suppression and allocation rules where no standard exists.
- **P6 stakeholders.** Table I.
- **P7 interdependence.** An error in bronze reaches the dashboard: counting by report date would create a false outbreak, which would raise forecast demand and move stock. Every gold row carries its bronze batch IDs, so such an error can be traced back.

B. Conflicts C1 to C5

C1 sensitivity against specificity. We allow at most two false alerts per district-year and take the most sensitive setting under that limit: EARS C2 at threshold 4.0 finds 33 of 35 outbreaks with 1.89 false alerts per district-year (Section VI).

C2 granularity against privacy. We publish district by week, not upazila. Every count from 1 to 4 is hidden, and a second cell is hidden in any row or column with only one hidden cell, so no total reveals it.

C3 efficiency against equity. The equity floor α gives every district at least α times its population share of each resource sent, capped at its own need, so $\alpha = 1$ is a population split that never sends a district more than it needs. The price is measured, not assumed: at a quarter or three quarters of the money needed a floor costs under one point of need met, but at half the money it costs 1.5 to 2.5 points, so the stated rule recommends $\alpha = 0$ (Section VI-C). The ministry can still choose a floor in the what-if panel and see its cost.

C4 freshness against cost. Forecasts and allocations are weekly decisions, so they run in a daily batch. Alerts can also run from a Structured Streaming file source that refreshes them as files land (Section IV-F); between files it holds almost no memory.

C5 accuracy against interpretability. The forecast is a negative-binomial GLM (NB-GLM) with seasonal terms and lagged rainfall and temperature: each coefficient can be read as a rate ratio. It beats the seasonal-naive baseline with a relative WIS of 0.84. We did not test a more complex model, such as gradient boosting or a hierarchical model that pools districts. The GLM already beats the baseline at every horizon, and a district officer can check its sign and size against what they see. The cost of that choice is that we cannot say how much accuracy interpretability gives up here; a pooled or boosted model in the same backtest is the comparison to run next.

III. Architecture and Design Decisions

EpiScope follows the brief's reference architecture: a medallion lakehouse (bronze, silver, gold) on Apache Spark 4.0.1 [3] in local mode with Delta Lake 4.0.1 [4], analytics jobs that read gold and write gold, a serving step that exports suppressed JSON, and a D3.js v7 [5] dashboard. One command, `make all`, runs every stage as a small DAG with retries (`reference`, `generate`, `bronze`, `silver`, `gold`, `analytics`, `serve`, `catalog`; Fig. 1); Java 17 and Python 3.12 come from `mise.toml` and every Python package is pinned in `requirements.txt`.

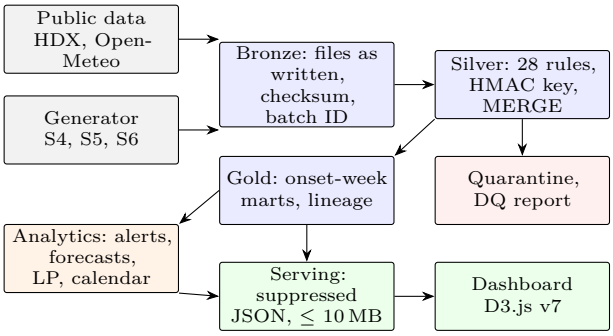


Fig. 1. EpiScope data flow. Blue: Delta lakehouse on Spark; one Makefile runs every stage.

Delta rather than plain Parquet. Late records (up to 21 days) must update their epi-week without duplicates. Delta's ACID MERGE makes that a single atomic statement, `replaceWhere` rewrites only the changed weeks of gold, and the transaction log gives time travel for "what did we know on date t". Iceberg would also work; Delta has the documented pairing with Spark 4.0.1 and pip packaging.

Local filesystem rather than HDFS or an object store (ING-5 not done). A Docker HDFS or S3 store costs 1–2 GB of RAM that Spark needs on the target laptop. All paths come from two environment variables, so the lake can move to another URI without code changes.

Partitioning. Bronze is partitioned by `batch_id`, because silver reads exactly one new batch at a time. Silver encounters and gold `cases_weekly` are partitioned by `epi_year` only: one year of gold holds at most $64 \times 4 \times 53$ rows, so a finer key (district or week) would create thousands of tiny files at SF=1 without saving any reads. The proposal planned `division` as a second key; we dropped it for the same reason.

Catalog. The jobs read and write by path, and a separate `make catalog` step registers every table as an external table (`bronze.encounters`, `gold.cases_weekly`, ...) in a Hive metastore stored inside the lake. Derby allows one process at a time, so keeping the catalog out of the jobs means a locked metastore can never stop the pipeline.

Orchestration (ENG-4). `scripts/dag.py` runs the stages in dependency order and retries a failed stage twice, 30 s and then 60 s later, before it stops; each run leaves a JSON record of attempts and times in the lake. Retrying is safe because every stage is idempotent: bronze skips files it has already loaded, silver merges and gold rebuilds only changed weeks.

TABLE III
Alternatives considered

Chosen	Rejected	Reason
Python DAG behind `make`	Airflow, Prefect	Airflow needs 4–8 GB of RAM; Prefect is a new dependency for eight stages
Delta Lake	Iceberg, plain Parquet	MERGE and time travel with documented Spark 4.0.1 support; Parquet has neither
Batch, plus a streaming refresh	Streaming only	Decisions are weekly; the stream (Section IV-F) reuses the batch code for alerts
Local filesystem	HDFS, MinIO	1–2 GB of RAM saved on a 16 GB laptop
Static JSON	An API server	Under 10 MB of aggregates; nothing to run or secure

IV. Data Generation, Quality, Lineage and Privacy

A. Public sources and the generator

District boundaries, codes and the 2022 census population by age band and sex come from the HDX COD-AB and COD-PS datasets [6]; daily precipitation, temperature and humidity at each district centroid for 2021–2025 come from the Open-Meteo archive [7]. Open-Meteo allows about 5,000 weighted calls an hour and 64 districts over five years cost about 8,400, so the download is throttled and the processed reference files (0.6 MB) are committed. A clean clone therefore skips a two-hour download.

The generator (`generator/`) writes the three synthetic sources: encounters (S4) as one CSV folder per report date, a register of 736 facilities (S5) and daily stock snapshots (S6) as JSON. It is vectorised numpy, seeded per (seed, stream, district, year), so the same arguments give byte-identical output whatever the number of worker processes. Rates are per head of population, so the row count follows the census, not a constant: SF=1 gives 34.1 million encounter rows for 2021–2025, planted defects included.

Each disease has its own annual cycle. Dengue peaks in early October, because OpenDengue's national counts peak in October in 2022 and 2024, later than "follows the monsoon" suggests. Dengue responds to rainfall and temperature four to eight weeks earlier, diarrhoea spikes after extreme rainfall, malaria is concentrated in the three Hill Tracts districts and Cox's Bazar, and every count has negative-binomial noise. Reporting delay is a discretised negative binomial with mean 3 days truncated at 21, and 65% of Friday and Saturday reports move to Sunday (Bangladesh's weekend). Each run injects 35 outbreaks of different shape and size, including a slow-growing one and a nationwide novel respiratory surge, and two reporting artefacts, each a facility that goes silent for three weeks and then back-fills. They are written only to `ground_truth/`, which the detection code never reads (a test fails if it does).

The generator also plants the defects the quality rules must catch: 2% repeated `encounter_id`, 0.5% orphan `facility_id`, 0.5% invalid age, 0.5% invalid sex, 0.3% malformed ICD-10 codes and 0.3% onsets after the report date. A 14-day incremental drop (`make drop`) uses the same layout and carries records with onsets up to 21 days before the drop.

B. Bronze: keep what arrived

`pipeline/bronze/ingest.py` finds files by glob, never by name, date or count. It computes each file's SHA-256 and skips any file whose checksum is already in the ingest log, so a re-run reads nothing and a new drop loads only its own files. Every value of the contract fields is kept as a string, exactly as written, and each row gets the source, its file, the checksum, the batch ID and the load time (ING-1). One run is one batch; batch IDs start with a UTC timestamp and therefore sort in load order.

C. Silver: rules, quarantine and de-identification

The rule catalogue (`pipeline/quality/rules.py`) has 14 rules for encounters and 7 each for facilities and stock (DQ-1). Each rule is a Spark column expression that is true when a row breaks it, with a stable reason code, a severity (quarantine or warn) and the requirement it serves. The six planted defects map one to one onto six rules:

- `DQ_DUP_ENCOUNTER` and `DQ_ORPHAN_FACILITY`;
- `DQ_INVALID_AGE` and `DQ_INVALID_SEX`;
- `DQ_ONSET_AFTER_REPORT` and `DQ_BAD_ICD10`.

The rest cover nulls, types, long delays, future dates and outcome values. Spark 4 runs in ANSI mode, where a plain cast of a bad string throws, so every rule reads values through `try_cast` and an unparseable value is a defect of its own. A failing row goes to `silver.quarantine` once, with the list of every code it broke. In the SF=1 load, 1,367,440 of 34,136,930 encounter rows (4.0%) were quarantined. Each load writes one row of rule pass rates to `gold.dq_report`, which the dashboard shows (DQ-2).

Before a clean row is stored, `patient_id` becomes an HMAC-SHA256 under a key held only in `.env` (PRV-1). Spark has no HMAC function and a Python UDF over 25 million rows is slow, so the HMAC is built from Spark's native `sha2` with the RFC 2104 inner and outer pads. A plain salted hash would not be enough: national IDs are few enough to enumerate, so the secret key is the only protection. The code refuses to run with a missing key or the placeholder from `.env.example`. Exact age becomes an age band.

D. Idempotent incremental loads (ING-4)

Clean rows are merged into `silver.encounters` with a Delta MERGE on `encounter_id`. A matched row is replaced only when the incoming `report_date` is later. That one condition gives both required properties: re-running a batch matches every row with an equal date and changes nothing, and a resubmitted or late record replaces the old copy and moves to its own onset week. The MERGE is limited to the epi-years in the batch (plus one either side); partition pruning then keeps it from scanning the whole table.

Silver records the weeks each batch changed: the onset weeks of new and replaced rows and the old weeks of replaced rows. Gold rebuilds only those weeks, plus the week after each, with a Delta `replaceWhere`.

E. Gold, lineage and patient de-duplication

`gold.cases_weekly` counts cases, admissions, deaths and lab confirmations per district, disease and ISO week of *onset*. Counting by onset rather than report date is what keeps the back-filled weeks of the silent facility from looking like an outbreak. Every district-week with at least one reporting facility gets a row for all four diseases, so a missing row means "nothing reported", not zero, which the detector uses to treat silent weeks as missing (AN-3). Each row carries the sorted bronze batch IDs of the records it counts, which link through the ingest log to file names and checksums (DQ-3).

Cross-facility patient de-duplication (DQ-4) flags a record as a repeat when the same patient key, disease, sex and compatible age band appear at another facility within seven days. At SF=1 it flagged 8,499 pairs, of which 8,493 were true repeats (precision 0.999). Recall within the window was only 0.11, because the generator draws age and sex afresh for a repeat visit. Most planted repeats fail the attribute check. We kept the strict rule: a false merge would hide a real case.

F. Time travel and the near-real-time path

Every gold build appends one row to `gold._snapshots`: the Delta version of the weekly case table it wrote and the latest report date that version contains. The `asof` module answers "what did we know on date t" by reading the newest version whose data ends on or before t with Delta's `versionAsOf` (ING-7). A test checks that the answer for a date between two loads leaves out a record that arrived 21 days late. A backtest origin must not see such a record either.

`pipeline/stream.py` (ING-6) is a Structured Streaming query over the encounter file source. Each micro-batch with new files runs the same incremental chain as the daily batch (bronze, silver MERGE, gold rebuild of the changed weeks, detection, export) in one Spark session. A streamed record therefore passes the same rules and keeps the same lineage. The bronze checksum log makes the stream and the batch safe to mix: whichever sees a file first loads it. We measured the time from the newest file of a 14-day drop to refreshed alerts twice at SF=1. With a 60 s trigger it was 301.5 s for 112 files (bronze 32 s, silver 118 s, gold 54 s, detection 32 s, export 22 s, the rest waiting for the trigger), 1.5 s over the 5-minute target. Since most of the miss was trigger wait, the default is now 30 s; the next drop of 80 files took 219.2 s (bronze 29 s, silver 96 s, gold 41 s, detection 25 s, export 18 s).

G. Hidden-drop drill

`scripts/hidden_drop_drill.sh` rehearses the instructor's test: clone, `make setup`, `make all`, a new 14-day drop with an unseen seed, the incremental steps, then the same steps again. On the final code (21 October, SF=0.05) it passed every check. The drop (seed 4675) became one new bronze batch, silver gained 8,888 rows, and the data moved from 31 December 2025 to 14 January 2026. The re-run changed nothing and left no duplicate IDs or gold keys. The export was 9.76 MB with no count from 1 to 4. The whole drill took 1,717 s (28.6 minutes), including `make setup`. Two inputs do not come from the clone's own steps: the drill copies data/reference from the working checkout and writes a random salt into `.env`. Both repeat what the README path already gives, because the processed reference data is committed and `make setup` writes a random salt.

V. Analytics Methods

A. Outbreak detection (AN-1, AN-3)

Both detectors run per district and disease on a series prepared in Spark (`analytics/detection/`). First, every week a district reported nothing at all is marked missing, as is any week where fewer than 80% of its expected facilities reported; above that, the count is scaled by 1/completeness. Second, counts are divided by a seasonal index per division, disease and week of year, taken as the median over the *other* years, so an outbreak cannot raise its own expected value.

EARS C2 [8] compares each week with the mean and standard deviation of lags 3 to 9 and runs on Spark window functions. CUSUM [9] accumulates standardised excesses over a 26-week baseline behind a two-week guard band and runs in `applyInPandas`. Neither may alarm unless the last four weeks hold at least five cases, which keeps near-empty series quiet. Farrington [10] was ruled out: it needs about five years of history before its first test week, and we have five in total.

The reporting artefact is handled twice: gold counts by onset week, so back-filled records land on the weeks they happened, and the silent weeks count as missing, not as zero.

B. Forecasting (AN-4)

For dengue and diarrhoea, one model per district and horizon $h = 1, \dots, 4$:

$$\log \mu_t = \beta_0 + \sum_{k=1}^{2} \mathrm{Fourier}_k(t) + \beta_r \, \mathrm{rain}_{t-\ell} + \beta_T \, \mathrm{temp}_{t-\ell} + \beta_y \log(1 + \mathrm{level}_{t-h}) \quad (1)$$

with lags ℓ of four to eight weeks and $\mathrm{Var}(y) = \mu + \alpha\mu^2$ (NB2 [11]), fitted with statsmodels. The baseline is the seasonal-naive forecast, the value of the same week a year earlier [12]. Because every weather lag is at least as long as the horizon, no weather forecast is needed. Districts are fitted in parallel through a cogrouped `applyInPandas`. Predictive quantiles are the plug-in NB quantiles, which ignore parameter uncertainty.

C. Demand, allocation and calendar (AN-6, AN-7, AN-9)

Forecast cases become demand for beds, IV saline, NS1 kits, ORS and antimalarials through admission shares estimated from the data, lengths of stay and units per case or admission, all in `demand_assumptions.py` and marked as our assumptions. The LP plans against the 90th percentile.

The allocation is the brief's programme solved with PuLP 3.3.2 and CBC, with three changes of ours. First, unmet need for each resource is divided by its national net need, so a bed and a test kit are not worth the same in one objective. Second, each district's unmet need is priced in four equal slices with weights 1, 2, 3 and 4, so the last quarter of a shortfall costs most. With a single flat price every unit of a resource is worth the same in every district; the LP is then indifferent between districts and returns a corner, with some districts fully supplied and the rest given nothing (Section VI-C). Net need is the four-week 90th-percentile demand minus current stock; for beds the patients already admitted are counted as holding a bed, so they are not counted twice. Third, no district receives more of a resource than its net need ($x_{d,r} \le \mathrm{need}_{d,r}$), and the equity floor is $x_{d,r} \ge \min(\alpha\, p_d \sum_e x_{e,r},\ \mathrm{need}_{d,r})$, where p_d is the district's population share. The minimum needs one binary variable per district and resource (need met in full, or the population floor binds), so the model is a small mixed-integer programme. $\alpha = 0$ is pure efficiency; $\alpha = 1$ means each district gets its population share of what is sent, but never more than it needs. The model is feasible for every α because $x = 0$ satisfies every constraint.

The preparedness calendar fits a seasonal-trend decomposition by loess (STL) [13] to each division and disease, inside `applyInPandas`. Onset is the first week after the trough where the seasonal index passes 25% of the trough-to-peak range; supplies are pre-positioned three weeks before onset and campaigns start four to six weeks before it.

D. Epidemic scenarios (AN-10)

A deterministic SEIR model [14] (`scenario/seir.py`) is fitted to the national respiratory surge. The surge is found without the ground truth: weekly ILI counts minus the median of the same week in the other years, keeping the run of weeks around the largest excess. With latent and infectious periods fixed at three and five days, least squares fits R_0, the seed and the share of infections that are reported. Scenarios then cut contacts by 20% or 40% from the first surge week, or two weeks later.

VI. Evaluation

Every number in this section comes from one clean SF=1 build (seed 42, real weather; 34.1 million encounters, of which 1.37 million, 4.0%, were quarantined) from a fresh clone of the final code, with the result files in `docs/experiments`. The exceptions are the scaling runs (Table VII) and the before-and-after measurements of the fixes, which say where they come from.

A. Detection

`make evaluate` scores every alert against the 35 injected outbreaks in `ground_truth/`, the only code allowed to read that folder. Consecutive alert weeks merge into one episode. An episode that overlaps no outbreak window counts as a false alert. The sweep picks, for each method, the most sensitive setting that keeps false alerts at or below two per district-year (conflict C1, Fig. 2). Table IV shows the chosen operating points.

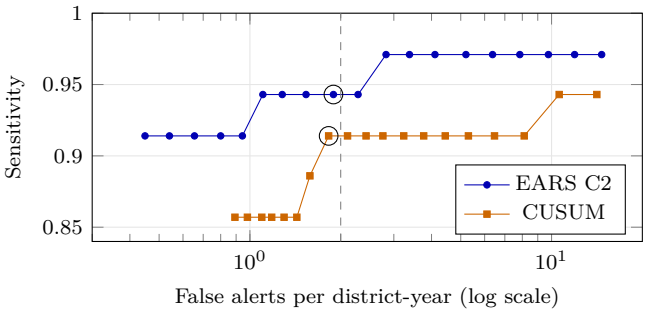


Fig. 2. Detection trade-off (conflict C1): every setting of the sweep, the two-alert limit dashed, the chosen settings circled.

TABLE IV
Detection at the chosen operating points

	EARS C2	CUSUM
Setting	threshold 4.0	$k = 0.5$, $h = 6.5$
Outbreaks detected	33 of 35	32 of 35
Sensitivity	0.94	0.91
False alerts per district-year	1.89	1.82
Positive predictive value	0.15	0.25
Median days to detection	8	9.5
Detected within 14 days	0.91	0.77

EARS C2 is the default: it finds one outbreak more and finds them sooner, at a similar false-alert rate. CUSUM's alerts are more often right (PPV 0.25 against 0.15). By shape, EARS C2 found 31 of 33 fast outbreaks, the slow one after 41 days and the nationwide surge after one day; CUSUM found 30 fast ones, the slow one after 69 days and the surge after one day. We expected CUSUM to win on slow growth. On our one slow outbreak it did not. Both methods missed the same outbreak, `OB42-017`. The threshold was tuned on planted outbreaks and should be re-tuned on real counts.

The reporting artefact (a facility that goes silent and then back-fills) is checked by a test: a naive detector on report-date counts alerts on the back-fill week, while ours, on onset-week counts with silent weeks missing, does not.

B. Forecast backtest

The backtest refits the per-district NB-GLM at 52 rolling origins (every fourth origin, reused in between) and forecasts one to four weeks ahead using only data up to each origin. The seasonal-naive baseline is scored on the same district, disease, origin and horizon cells. The weighted interval score [15] uses eleven central intervals plus the median; relative WIS below 1 means the model beats the baseline. MAE is the mean absolute error of the median forecast.

TABLE V
Backtest over 52 origins, model against the seasonal-naive baseline

Disease	h	MAE	MAE base	Cov. 80%	Rel. WIS
Dengue	1	155	164	0.80	0.88
	4	161	174	0.79	0.90
Diarrhoea	1	67.5	96.2	0.79	0.70
	4	70.5	96.3	0.79	0.74
All	1–4	113	133	0.79	0.84

The model beats the baseline at every horizon for both diseases, by more for diarrhoea (relative WIS 0.72) than for dengue (0.89). Its 80% intervals cover 79% of outcomes, against 77% for the baseline, so the bands mean what they say. For dengue, 105 of the model's WIS of 130 is underprediction: the model is slow to follow a season's rise. That matters for stock, and it is why the plan uses the 90th percentile, not the mean.

C. Allocation against the population baseline

The first allocation runs were degenerate: generated stock was 3 to 225 times the four-week demand, so almost nothing needed allocating, and the LP sent what little there was to two districts. `allocation_diagnosis.md` traces three faults: stock that ignored the case load, an objective that valued every district equally (so the solver returned a corner: in a test with need everywhere, 21 districts got all their beds and 42 got none), and beds counted twice. After the fixes, at SF=0.25 and half of the money needed, all 64 districts receive supplies, the five largest receive 22% of the value instead of 100%, and the Gini of per-capita spend falls from 0.98 to 0.27.

At SF=1 the national net need is 43,558 litres of IV saline, 513,721 ORS sachets and 691 antimalarial courses; NS1 kits and beds need no top-up, because the data end in December, after the dengue season. Covering all of it costs 7.78 million BDT (B_{ref}). Table VI compares the LP with the population-proportional baseline at five budgets.

TABLE VI
Share of need met, LP ($\alpha = 0$) against population split

Budget ($\times B_{\text{ref}}$)	0.25	0.5	0.75	1.0	1.5
LP (%)	50.5	67.8	82.7	100	100
Population split (%)	17.6	34.4	50.1	63.1	70.9

The LP meets about twice the share of need at every scarce budget. The efficiency and equity curve (Fig. 3) gives the price of equity.

The floor is capped at need: a district gets at least $\min(\alpha\, p_d \sum_e x_{e,r},\ \text{need}_{d,r})$ and never more than its need, so $\alpha = 1$ means a population share of what is sent, but never more than a district needs. No scenario now sends any district more than its need (`excess_cost` is 0 throughout). At a quarter and three quarters of B_{ref} every floor costs less than one point of need met. At half the budget a floor from 0.1 to 0.9 costs 2.5 points (65.3% against 67.8%) and $\alpha = 1$ costs 1.5, so the rule recommends $\alpha = 0$. The tranche prices already spread the money: at $\alpha = 0$ and half the budget the worst-covered district gets half of its need. The floor does not improve on that. At half the budget, $\alpha = 0.1$ lowers the worst-covered district from 0.50 to 0.375 of its need and moves the Gini of per-capita spend from 0.261 to 0.271, and at no budget does any α raise the worst-covered district by more than 0.003 over its $\alpha = 0$ level (`allocation_curve.csv`). For policy, the floor in this model buys a population-based guarantee, not better coverage for the worst-off; a ministry that wants the latter should raise the weight of the deepest shortfall instead.

D. Epidemic scenarios

The SEIR fit to the nationwide respiratory surge (nine weeks from 28 October 2024) gives $R_0 = 2.04$ with $R^2 = 0.91$; the generator used 1.8. Cutting contacts by 20% from the first surge week lowers the peak by 46% and the cases over the surge and the following 12 weeks by 18%; a 40% cut lowers them by 87% and 63%. Starting the 40% cut two weeks late still lowers the peak by 80% but the cases by only 53%, which is the case for acting on the first alert (`seir_scenarios.json`).

E. Preparedness calendar

To check the calendar (AN-9) against what happened, we applied its onset rule to each year's observed weekly cases per division (three-week moving average) and compared the year's onset with the calendar's. For dengue the median difference is 0 weeks and 39 of 40 division-years fall within two weeks; for diarrhoea and malaria the median difference is one week, with 85% and 87% within two weeks (`calendar_check.json`, run on the SF=0.25 build with the same seed, so the same seasonal timing). We did not publish an ILI calendar check: its yearly onset spreads widely between years (standard deviation 4.6 to 16.5 weeks by division), which the nationwide surge distorts.

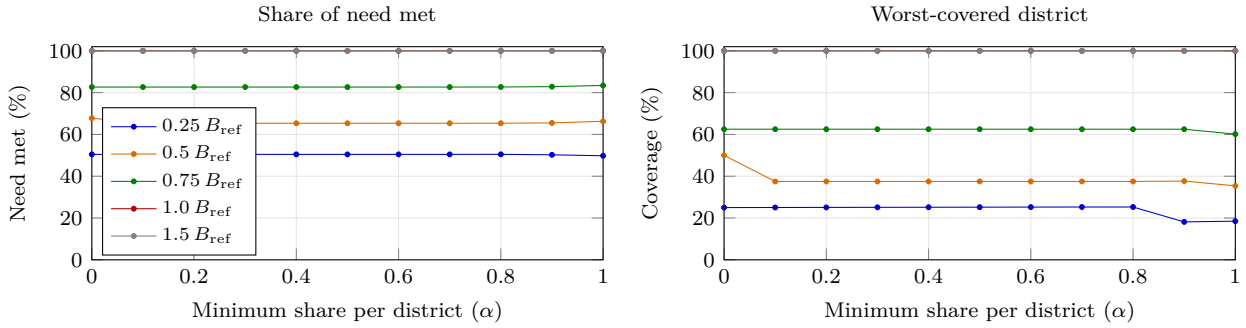


Fig. 3. Efficiency and equity curve (AN-8) from `allocation_curve.csv`: share of need met and the worst-covered district's share of its own need against the equity floor α, one line per budget.

F. Scalability and Spark optimisations

`scripts/scaling.sh` builds each scale factor from a fresh clone and times every stage (Table VII). The laptop has an Intel Core i5-8350U (4 cores, 8 threads) and 15.4 GB of RAM; Spark runs as `local[4]` with a 5 GB driver. The three scaling runs (20 and 21 October, commits 89eea98, 01534b6 and f1fc38d) predate the allocation fix and the second optimisation below; the last column is the final SF=1 build, also from a fresh clone (`sf1_final_run.json`). SF=1 finishes in 25 to 28 minutes, inside the 45-minute gate. Silver grows with the row count and is the slowest stage. Analytics hardly grows, because it works on weekly series per district, not on rows. A first SF=1 attempt failed in silver when Spark's spill filled the small RAM-backed `/tmp`; spill now goes to a disk folder.

TABLE VII
Minutes per stage from a fresh clone

	SF=0.25	SF=0.5	SF=1	SF=1 final
Encounter rows (M)	8.5	17.1	34.1	34.1
Generate	0.2	0.4	0.6	0.6
Bronze	1.9	2.8	4.3	4.3
Silver	3.8	6.7	11.1	11.0
Gold	1.8	2.3	3.5	3.3
Analytics	6.9	7.1	8.0	4.9
Serve	0.7	1.1	0.9	0.9
Total	15.4	20.4	28.3	25.0
Raw data (GB)	1.19	2.16	4.09	
Bronze (GB)	0.19	0.35	0.67	
Silver (GB)	0.77	1.47	2.88	
Gold (MB)	2.0	2.0	2.1	
Export (MB)	9.7	9.8	9.8	

Two optimisations were measured, each with its outputs checked unchanged.

- **Silver evaluates the quality rules once.** The 14 encounter rules, including the duplicate-ID rule that sorts the whole load, were evaluated once for every output (quarantine, its count, the quality report and the clean MERGE). They are now evaluated once and persisted. At SF=0.25 silver's Spark time fell from 325 s to 214 s, and every silver table kept the same row count and row hash (`silver_optimisation.json`). A fresh run for the Spark UI gave the same ratio: 6.3 minutes before, 4.2 after (Fig. 4, top).
- **The backtest uses every core.** The Spark UI showed the backtest's model stage running as a single task for 5.1 minutes (Fig. 4). Its input is only a few megabytes, so adaptive query execution had merged the shuffle into one partition and fitted all 64 districts one after another. Keeping one partition per district for that job gives 64 tasks. At SF=0.25 the job fell from 325 s to 165 s with identical scores (`backtest_optimisation.json`).

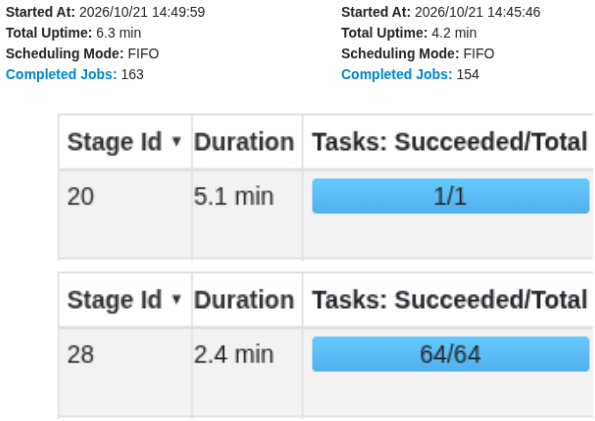


Fig. 4. Spark UI evidence. Top: the silver job before (left) and after (right) evaluating the rules once. Bottom: the backtest model stage, one task for 5.1 minutes before and 64 tasks for 2.4 minutes after. Full screenshots: `report/figures/`.

VII. Dashboard Design and Usability

The dashboard (`dashboard/`) uses only D3.js v7 and the TopoJSON client, both vendored with subresource-integrity hashes. It loads only the aggregated JSON exports, which must stay under 10 MB: the export keeps the most recent weeks that fit. Five views are linked (Fig. 5): a district choropleth of incidence per 100,000 with alert markers, a weekly time series with a brush, forecast and bands, a

seasonal heatmap, an allocation view with a what-if panel, and a ranked early-warning table. Suppressed cells are hatched and every view shows the data-freshness time. **Design rationale.** Incidence uses the viridis scale, which stays ordered in greyscale and for the common colour-vision deficiencies; alerts use the Okabe-Ito vermillion, which stands out against it. Counts are shown per 100,000 on the map, so a large district does not look worse only for being large, but as raw cases on the time series, which is what a store-keeper plans with. Suppressed cells are hatched, not left blank, so "hidden" never reads as "zero". Every forecast carries its 80% band. District by week is the finest level shown: it is what an officer acts on and the coarsest level that still locates an outbreak. Below 560 px the views stack in one column for a phone.

What-if panel (VIS-5). Three sliders recompute the views in the browser from precomputed results. The budget and the minimum share per district pick one of the 55 allocation scenarios; the alert-level slider multiplies each method's threshold from 0.5 to 2.0, so an officer sees how many alerts a stricter or looser level would give before anyone changes the detector.

District view (PRV-3). Opening the dashboard with `?district=` and a district code loads that district's bundle: its own series, alerts, forecast and allocation in full, and the rest of the country drawn lighter as division totals.

Simulated persona walkthroughs. We had no access to non-CS users in the final phase, so the usability check was run as *simulated persona walkthroughs with AI agents*: three AI agents, each briefed as one persona, used the real dashboard of the final SF=1 build through a scripted headless browser. They are not human participants, and their findings are a design review, not user evidence. The personas were a district health officer in Cox's Bazar using the district view, a hospital store-keeper in Chattogram, and a senior ministry official. Each did five tasks: find this week's alerts, read a forecast with its range, explain an allocation, use the what-if panel, and read the policy brief (protocol and records in `docs/usability/`).

TABLE VIII
Tasks completed by each simulated persona (of five)

Done / partly / not done	Officer	Store-keeper	Ministry
Round 1	0 / 5 / 0	0 / 5 / 0	0 / 4 / 1
Round 2	4 / 1 / 0	5 / 0 / 0	4 / 1 / 0

Round 1 found 48 problems. Nothing showed who was on alert *now*: the map and the table counted alerts over six months. The newest week, still incomplete, looked like a sudden fall. The four-week forecast was a flat line with no numbers, and a threshold line drawn on the case axis contradicted the alerts. The allocation used codes such as q90 and α, gave no units or currency, and opened on a supply with nothing to allocate. We added a line naming the districts on alert in the last full week, marked the partial week, added a forecast table with a cautious four-week total, removed the threshold line, renamed every label in plain words, and added a sentence explaining each district's allocation. In the confirm round the agents rated 26 of the 48 problems fixed, 20 partly fixed and 2 not fixed (Table VIII). Round 2 logged 17 new problems, 13 of them distinct, two of high severity: the hollow alert markers were drawn solid, and at the largest budget with $\alpha = 1$ the plan sent 9,237 antimalarial courses against a national shortfall of 684, because a full population split forces supplies to every district. We then fixed the markers, called a cell hidden to protect another cell "hidden" instead of "fewer than 5", made the ranking fit its panel, added a cautious four-week total and an all-supplies line, and said on screen that the plan does not move stock between districts. These fixes were not tested in a third round. The over-allocation was then removed by capping every allocation at need (Section VI-C). Still open: unlabelled budget lines in the equity chart, no delivery week, and method names that remain partly technical.

VIII. Ethical, Societal and Sustainability Considerations

Privacy by design. No real patient data is used; every encounter is synthetic and labelled so. Direct identifiers stop at bronze: silver and gold hold only the keyed hash and age bands, the key never enters Git, and every export hides counts from 1 to 4 with complementary suppression, so a total cannot reveal a hidden cell. One residual risk is known and written in the code: the padded HMAC key appears in Spark's query plan, so Spark event logs of the silver job must be treated as secret and kept outside the repository.

Record-level extracts and public release (PRV-4). `pipeline/privacy/kanon.py` measures k-anonymity [16] on any record-level extract. The quasi-identifiers are age band, sex, district, onset week, disease and facility. k-anonymity does not stop a homogeneity attack, where everyone in a class shares one diagnosis, so a disease extract also needs l-diversity. For a public release, differential privacy [17] is the stronger tool: the Laplace mechanism adds noise with standard deviation $\sqrt{2}/\varepsilon$ to each count, about 1.4 cases at $\varepsilon = 1$. The privacy budget adds up over every published cell and week, and noisy counts can be negative and must be post-processed. We would use it for the public release only; ministry users keep exact counts with suppression. **Role-based access (PRV-3).** A district user receives a bundle with their own district in detail and every other area only as a division total. A division total is published only if every district count under it is visible in the national view. Subtracting one published number from another can then never reveal a hidden count. An automated security review of the first version found such a leak, and this rule closes it. The bundles decide what each role is sent, but a static web server cannot stop a district



Fig. 5. The dashboard on the final SF=1 data. Left: the "on alert now" line, the map (solid triangle: alert in the last full week), the weekly series with its forecast table, and the seasonal heatmap. Right: the allocation panel with the what-if sliders and the cost of a minimum share.

user from requesting the national files: a real deployment needs a login and a separate path per role.

Equity. The equity floor gives every district a population share of what is sent, but never more than it needs. A district that shows no need gets nothing, so the floor cannot protect a district whose reports are missing. The pipeline keeps such gaps visible instead of hiding them: a district-week in which fewer than 80% of the expected facilities reported is marked missing, not zero, and the dashboard shows it as "no report". It does not raise an alert on its own; a reporting-gap alert is left for future work. The floor costs efficiency, and Section VI-C gives the price. The weights in the objective are a value judgement that the ministry, not the analysts, should own.

Misuse and trust. A named district on an alert list can be stigmatised, and too many false alarms teach officers to ignore the list; the operating point was chosen to keep false alerts to about two per district a year for that reason. Big-data signals can drift away from the truth when behaviour or reporting changes [18], so the alerts are a prompt to investigate, not a diagnosis.

Footprint. The whole pipeline runs on one 15 GB laptop in under 30 minutes at full scale, with no cluster, cloud service or GPU, which keeps both the energy use and the running cost low enough for a district office to repeat.

IX. Limitations and Future Work

The data is synthetic, so detection and forecast scores measure how well the methods recover patterns we planted, not real-world accuracy. The storage layer runs on the local filesystem, not HDFS or an object store (ING-5). Patient de-duplication is precise but misses most repeats whose age or sex was recorded differently.

- **Data.** Synthetic stock follows a simple min/max policy; real stock records would change the allocation. Next: load a real stock feed through the same S6 contract.
- **Detection.** The threshold was tuned on outbreaks we planted, and a district that stops reporting raises no alert. Next: re-tune on reported outbreaks from DGHS bulletins, and add a reporting-gap alert.
- **Forecasting.** The intervals ignore parameter uncertainty, and dengue forecasts tend to fall short in a surge. Next: a bootstrap over the fit and a growth term.
- **Allocation.** Costs and units per case are rough assumptions, and the LP plans one four-week round. Next: real prices and a rolling plan that carries stock forward.
- **Dashboard and access.** Role bundles are not enforced by the static server. Next: a login with a path per role.
- **Scenarios.** The SEIR model is deterministic and national. Next: per-division fits with a range.

References

[1] World Health Organization, "Dengue: Bangladesh, disease outbreak news DON498," https://www.who.int/emergencies/disease-outbreak-news/item/2023-DON498, 2023.

[2] ——, *Technical Guidelines for Integrated Disease Surveillance and Response (IDSR)*, 3rd ed. WHO, 2019.

[3] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, 2016.

[4] M. Armbrust *et al.*, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, 2020.
[5] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, 2011.
[6] OCHA ROAP and Bangladesh Bureau of Statistics, "Bangladesh subnational administrative boundaries (COD-AB) and population statistics (COD-PS)," Humanitarian Data Exchange, https://data.humdata.org/dataset/cod-ab-bgd, 2025.
[7] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, https://open-meteo.com, 2023.
[8] L. Hutwagner, W. Thompson, G. M. Seeman, and T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," *Journal of Urban Health*, vol. 80, no. 1, 2003.
[9] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
[10] C. P. Farrington, N. J. Andrews, A. D. Beale, and M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," *Journal of the Royal Statistical Society A*, vol. 159, no. 3, 1996.
[11] J. M. Hilbe, *Negative Binomial Regression*, 2nd ed. Cambridge University Press, 2011.
[12] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.
[13] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
[14] W. O. Kermack and A. G. McKendrick, "A contribution to the mathematical theory of epidemics," *Proceedings of the Royal Society of London. Series A*, vol. 115, no. 772, pp. 700–721, 1927.
[15] J. Bracher, E. L. Ray, T. Gneiting, and N. G. Reich, "Evaluating epidemic forecasts in an interval format," *PLoS Computational Biology*, vol. 17, no. 2, 2021.
[16] L. Sweeney, "k-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, 2002.
[17] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference (TCC)*, ser. LNCS, vol. 3876, 2006.
[18] D. Lazer, R. Kennedy, G. King, and A. Vespignani, "The parable of Google Flu: traps in big data analysis," *Science*, vol. 343, no. 6176, 2014.

Appendix A
Policy Brief for the Director

To: Director, Disease Control & Preparedness Unit, Ministry of Health. **About:** what the EpiScope prototype can do for weekly outbreak warning and supply planning, and what to decide.

Where we are

EpiScope is a working prototype that watches all 64 districts every week for four diseases: dengue, diarrhoea, influenza-like illness and malaria. It warns when a district's cases rise more than expected, forecasts the next four weeks with a likely range, turns those forecasts into the supplies hospitals will need, and proposes how to share a fixed budget between districts. It also gives each division a calendar of when to prepare for each disease season. It was tested on realistic made-up patient records (34 million visits over five years) built on Bangladesh's real population and weather, because no real patient data was used. One ordinary laptop processes the whole country in about 25 minutes.

Three recommendations

1. Act on the weekly warning list. In testing, the list caught 33 of 35 planted outbreaks, half of them within eight days. The price is false alarms: about two a year in each district. Each district should expect to check a couple of warnings a year that turn out to be nothing, and should check every one, because a missed outbreak costs far more. For a fast-spreading respiratory virus, acting on the first warning matters most: in our model, cutting contacts by 40% from the first warning prevents about 63% of cases, against 53% if the same step comes two weeks later.

2. Send supplies where the need is, not by population. When there is only half of the money needed, sharing supplies by the measured shortfall in each district covers 68% of the shortfall, averaged over the supplies; sharing the same money by population covers 34%. The dashboard shows each supply separately, so a single supply can sit above or below that average. The dashboard lets you set a minimum share for every district (the equity floor): each district then gets at least that fraction of its population share of what is sent, but never more than it needs, so no supplies go where there is no shortfall. At a quarter or three quarters of the money a minimum share costs less than one point of the shortfall covered; at half the money it costs 1.5 to 2.5 points. This is a policy choice, and the dashboard shows its cost before you make it.

3. Prepare before each season, not during it. The calendar gives each division a week to have supplies in place and a week to start prevention work (table below). For dengue in Dhaka division, supplies should be in place by mid-July and the prevention campaign should start in late June, three and six weeks before the season usually begins in early August. Malaria matters mainly in Chattogram division (the three Hill Tracts districts and Cox's Bazar): supplies should be in place there by 4 May, with the campaign from 13 April.

Decision requested

Approve the supply plan for the next four weeks. Meeting every district's expected shortfall costs about 7.8 million taka. With the full amount, every shortfall is covered. With half of it (3.9 million taka), the need-based plan covers 68% of the shortfall (averaged over the supplies) and a population-based split 34%. Decide also whether every district gets a guaranteed minimum share (its population share, never more than it needs): at half the money the recommended setting is no minimum, because a minimum lowers the share of need covered by 1.5 to 2.5 percentage points.

When to act in 2026 (supplies in place by, prevention campaign started by):

Division	Dengue Supplies	Dengue Campaign	Diarrhoea Supplies	Diarrhoea Campaign
Barishal	13 Jul	22 Jun	12 Jan	5 Jan
Chattogram	6 Jul	15 Jun	26 Jan	19 Jan
Dhaka	13 Jul	22 Jun	26 Jan	19 Jan
Khulna	13 Jul	22 Jun	26 Jan	19 Jan
Mymensingh	13 Jul	22 Jun	19 Jan	12 Jan
Rajshahi	6 Jul	15 Jun	26 Jan	19 Jan
Rangpur	6 Jul	15 Jun	2 Feb	26 Jan
Sylhet	13 Jul	22 Jun	26 Jan	19 Jan

How to read the dashboard

The line at the top lists the districts on alert in the last full week. Choose a disease below it. The map shades each district by cases per 100,000 people; a solid triangle marks a warning in the last full week, an outlined one a warning earlier. Click a district to see its weekly cases, its forecast for the next four weeks as a table of expected cases and a likely range (eight times in ten the real number falls inside it), and its place in the warning list. The newest week is drawn as an open dot because its reports are not all in yet. The allocation panel shows how the chosen budget is shared; move the sliders to see what a larger budget or a higher minimum share would do. District officers open a view of their own district, with the rest of the country shown only as division totals.

How far to trust it

The test data were made up, so these results show that the methods work on realistic patterns, not that they will match real outbreaks to the day. The forecasts tend to fall short when dengue is rising fast, so stock is planned on the high end of the range, not the middle. Supply costs and the amount used per patient are our estimates and should be replaced with ministry figures. A warning is a reason to investigate, not proof of an outbreak.

How patient data are protected

Names and ID numbers never leave the secure store: each patient is replaced by a code that cannot be turned back into the number, and ages become age groups. Only totals reach the dashboard, and any count from one to four is hidden, together with any other number that could reveal it. A district officer sees their own district in detail and the rest only as division totals.

Appendix B
Contribution Matrix

Roles follow the proposal: Nishadul Islam Shezan led data engineering, Tahsin Ahmed Rafi analytics, Rimjhim Dey visualisation, and Samira Sultana platform and evaluation. Tahsin Ahmed Rafi, Rimjhim Dey and Samira were away during the final phase of the project, and Shezan implemented the remaining parts of their areas.

TEAMMATE: All: agree and sign

Fill the empty cells: each member's support (S) outside their lead area, the video row, the agreed shares (total 100), the commit and line counts from `git shortlog -sn` at the final tag, and signatures.

TABLE IX
Contribution matrix (L lead, S support, blank none)

Work area	Shezan	Rafi	Rimjhim	Samira
Problem analysis, proposal, conflicts	S	S	S	L
Generator and public data	L			
Ingestion, lakehouse, incremental loads	L			
Data quality, lineage, privacy controls	L		S	S
Outbreak detection and evaluation	S	L		S
Forecasting, backtest, demand	S	L		S
Optimisation and calendar	S	L		
Serving layer and D3 dashboard	S		L	
Scalability and Spark optimisations	S			L
Report and policy brief	S			L
Video production				
Agreed share σ_i (%)				
Commits / lines changed				
Signature				

Appendix C
Individual Reflections

Nishadul Islam Shezan

TEAMMATE: Shezan

Half a page in your own words.

Tahsin Ahmed Rafi

TEAMMATE: Tahsin Ahmed Rafi

Half a page in your own words.

Rimjhim Dey

TEAMMATE: Rimjhim Dey

Half a page in your own words.

Samira Sultana

TEAMMATE: Samira

Half a page in your own words.